

Small Models, Big Claims: Choosing and Quantizing Local LLMs on an Integrated GPU

An independent, single-machine measurement study

Daniela Zaragoza Rodriguez

orcid.unaudited840@passmail.com · @dzaragoza.bsky.social · ORCID: [0009-0003-8529-2638](https://orcid.org/0009-0003-8529-2638)

DOI: [10.5281/zenodo.22855666](https://doi.org/10.5281/zenodo.22855666)

Abstract

Running large language models locally on integrated graphics is now practical, but the community's model-selection advice is dominated by vibes: "just grab a 2B at Q4." This report presents a controlled, single-machine measurement study of twelve model/quantization configurations across three model families (Qwen2.5-1.5B, glm-edge-1.5b, SmolLM2-1.7B) on an AMD Radeon integrated GPU with the llama.cpp Vulkan backend. We calibrate a simple predictive formula for token-generation speed (throughput $\approx$ effective memory bandwidth $\div$ model file size), validate it against live interactive sessions, and measure ARC-Challenge accuracy at $n=800$ for every configuration. Three findings generalize across families: (1) generation speed follows file size with a family-dependent error of $\pm 10\text{--}15\%$; (2) quantizing from Q4 to Q6 recovers a consistent $+2.5$ to $+3.2$ percentage points of ARC accuracy in all three families; and (3) K-quant encodings (Q5_K_M, Q6_K) decode $7\text{--}11\%$ slower than legacy encodings (Q5_0) at equal size, with no compensating accuracy gain. We also show that one popular 2025 model (SmolLM2) scores ~ 20 points below its published benchmark impression under a strict letter-answer protocol, illustrating the hazard of comparing paper numbers across evaluation harnesses. We conclude with a reader-tunable decision rule replacing the author's own 20 tokens/second comfort threshold.

1. Introduction

1.1 Why this study

This study began with a concrete situation, not an abstract question. The author — a former researcher, on holiday, curious whether the local-LLM hobby had caught up to modest hardware — had one machine to work with: a small business desktop with an AMD APU whose graphics are integrated, budget-class, and five years old. This is the computer many people actually have. Nearly all published LLM benchmarking happens on datacenter GPUs or recent discrete GPUs; the guidance that filters down to users of ordinary hardware is folklore.

Three folklore claims motivated the study directly:

1. "*Just grab a 2B model at Q4*" — quantization format is treated as a detail, when it affects both speed (§5) and accuracy (§4).
2. "*Newer models are better*" — the study ended up testing a 2024 model against a 2025 one at equal size, and the older one won decisively on our protocol (§7).
3. "*The GPU is too weak to matter*" — we measured this (§1.3) and the folklore is half right: the iGPU is slower at generation, but twice as fast at prefill, and prefill is what you feel when a chat responds.

The questions we set out to answer:

1. Can generation speed be predicted from first principles (memory bandwidth ÷ file size), and how large are the deviations?
2. What does quantization *cost* in accuracy, and what does stepping up from Q4 to Q5 or Q6 buy back?
3. Do quantization *encodings* (legacy vs K-quant) affect speed or accuracy at equal size?
4. Which configuration maximizes accuracy subject to an interactive-speed constraint?

The answer to (4), up front: **Qwen2.5-1.5B-Instruct at Q5_0** — 75.2% on our ARC-Challenge protocol at roughly 21 tokens/second live, in a 1.17 GiB file. The rest of this report explains how we got there and, more importantly, how a reader with a different speed preference can redo the arithmetic for themselves.

1.2 Why this hardware

Because it is what non-enthusiasts own, and nobody benchmarks it. An integrated GPU on a shared-memory bus is the *hardest* case for LLM inference: no dedicated VRAM, bandwidth shared with the CPU, and a GPU architecture with no matrix-acceleration hardware (§2.1). If a simple predictive formula and a set of selection rules hold on this machine, they hold a fortiori on anything faster. The machine is also the author's daily driver, which means every recommendation in §9 was validated in real use, not just in the bench.

1.3 Why the Vulkan backend (and not CPU)

The obvious alternative to iGPU inference is CPU-only inference, and we measured both. The result split cleanly by phase:

- **Prefill (prompt processing):** the Vulkan/iGPU path is **~2× faster** than the 8 Zen 3 CPU cores (e.g., Qwen2.5 Q4_0: 322 vs 157 tok/s at pp128; 256 vs 153 at pp2048). Prefill is compute-bound, and the iGPU's shader array wins it.
- **Generation (tok/s):** the CPU is faster — Qwen2.5 Q4_0: 41.2 t/s CPU vs 28.4 t/s Vulkan (−31%).

We chose Vulkan anyway, deliberately: in interactive chat, time-to-first-token — dominated by prefill — is the responsiveness you feel on every message, while a 30% generation-speed differ-

ence changes how long you wait at the *end* of an answer. Put another way: we accepted slower last-words for twice-as-fast first-words, and then solved the generation-speed problem by model selection instead (which is §3–§6 of this report). The size budget derived in §3 is what makes that trade manageable.

A hardware note for replicators: all Vulkan measurements in this report were taken with a **2 GB BIOS UMA frame-buffer reservation**. During testing we discovered the machine had never actually run in the intended low-reservation (512 MB) condition — an early configuration confusion, corrected in the log — so every Vulkan figure, from the CPU comparison above to the benches of §3–§5, reflects the same 2 GB reserved state. The CPU-vs-Vulkan split above (2× pre-fill advantage, 31% generation cost for Qwen2.5 Q4_0) is therefore the *reserved* comparison; un-reserved Vulkan is expected to be strictly worse for generation, not better.

1.5 Use of AI in this work

This study was conducted in collaboration with an AI assistant, and the drafting of this report made substantial use of AI tools. Readers who consider that disqualifying should stop here; for everyone else, the exact division of labor:

What the human author did: chose the research questions, owned every methodological decision, ran every measurement on the physical machine (the author's only computer), curated the model roster, adjudicated all discrepancies between data and interpretation, and approved every claim in this text. The study exists because of her judgment calls — including the ones documented in §8, where her skepticism overrode the assistant's initial analysis.

What the AI assistant did: served as a tireless analytical collaborator — proposing experiment designs, making written predictions before measurements (several of which were wrong, preserved in §8), drafting analysis text and prose, and maintaining the lab notebook. Every number in this report originates from a measurement executed by the author; none was generated by the AI.

Why we disclose at this level of detail: because "made with AI" is uninformative. The scientifically relevant facts are that (a) all data is human-collected and machine-measured, (b) all interpretations were human-adjudicated, and (c) the AI's analytical errors are documented with the same rigor as its correct predictions — the correction log in §8 is partly a log of *the assistant being wrong and the author catching it*. That structure, we would argue, is the defensible way to use AI in empirical work: predict on record, measure, grade honestly.

1.6 A note on the assistant model

The assistant used was GLM, a large language model served on Mistral AI's Vibe platform. We name it for completeness and reproducibility, not endorsement — the same way one names an instrument's manufacturer. No claim is made that this model is uniquely capable of the collaboration described here; the methodology (pre-registered predictions, human adjudication, a mandatory corrections log) is deliberately model-agnostic, and we suspect the study's quality owes more

to that discipline than to any particular model. Correspondence regarding the model choice is, respectfully, out of scope.

1.7 Scope and honesty statement

This is a single-machine, single-benchmark, single-author study conducted during a vacation. It makes no claims of statistical power beyond what is stated, includes two predictions the author's AI collaborator got badly wrong (documented in §8), and confines its conclusions to the hardware, backend, and models listed in §2. Its value is in careful measurement and honest error accounting, not breadth.

2. Methodology

2.1 Hardware and software

Component	Detail
System	Lenovo ThinkCentre-class small-form-factor business desktop
CPU	AMD Ryzen 7 PRO 5755GE — 8 cores / 16 threads, Zen 3 ("Cezanne" APU family), 3.2 GHz base / ~4.6 GHz boost, 16 MB L3, AVX2 (no AVX-512), 35–65 W GE class
iGPU	AMD Radeon Graphics (Vega 8-class, GCN 5th-generation architecture) — integrated, unified memory (UMA), fp16 capable
Memory	64 GB DDR4-3200, dual channel (~51.2 GB/s theoretical peak, shared between CPU and iGPU)
Backend	llama.cpp Vulkan, build b10964 (commit b29c606e2)
Bench tool	llama-bench with <code>-t 8 -p 0 -n 128 -r 3 -ngl 99</code>
Evaluation harness	Custom <code>strict-arc.py</code> , ARC-Challenge test, n=800
OS	Windows 11 (PowerShell)

Where you have met this GPU. The PRO 5000G "Cezanne" APUs (including the 5755GE) shipped in business desktops and mini-PCs from 2021 onward and remain common in corporate fleets; the same Vega-architecture iGPU (in 6–8 CU configurations) appears across AMD's 4000–5000-series laptop and desktop APUs of 2020–2022. Millions of these machines are in circulation, and they are precisely the class of hardware that local-LLM advice ignores.

Why this GPU is the hard case. The Vulkan device report is explicit: `matrix cores: none , int dot: 0`. The Vega iGPU predates every hardware acceleration feature that modern LLM serving assumes — no tensor/matrix cores (which Nvidia tensor cores and AMD RDNA3/CDNA

matrix units provide), no integer dot-product support (DP4a, which some quantized kernels exploit). All matrix math runs through plain shader ALUs in fp16. Two consequences: (1) results here are a *floor* — any GPU with matrix cores will do better; (2) because the iGPU is a UMA device reading the same DDR4 bus as the CPU, model file size translates almost directly into memory traffic per token, which is what makes the §3 formula work so cleanly.

2.2 The models

Readers should not need prior familiarity with any of these. Three families were tested, chosen to span release years, design philosophies, and popularity in the local-LLM community:

Qwen2.5-1.5B-Instruct (Alibaba's Qwen team, late 2024). Part of the Qwen2.5 series — a broad family of instruction-tuned LLMs trained on 18 trillion tokens, released in sizes from 0.5B to 72B. The 1.5B is one of the most-downloaded small instruct models in the GGUF ecosystem and was the study's reference point for "what the 2024 state of the art looks like at this size." Notable for this study: a large (152k) vocabulary, which we later implicated in its family-specific speed penalty (§3.1). Technical report: [arXiv:2412.15115](https://arxiv.org/abs/2412.15115). Weights: [Qwen/Qwen2.5-1.5B-Instruct-GGUF](https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF).

glm-edge-1.5b-chat (Z.ai / Zhipu AI, 2025). Part of the GLM-Edge line, explicitly designed for consumer-class devices (phones, laptops, browsers). A representative of the "2025 edge-native" generation: newer training, edge-first design targets. Its official GGUF repository is unusually complete — it ships the full quantization ladder from Q4_1 through Q8_0, which is what made our encoding grid possible without self-conversion. Technical report: GLM-Edge (see the [official model page](#) for the technical report link). Weights: zai-org/glm-edge-1.5b-chat-gguf.

SmolLM2-1.7B-Instruct (HuggingFace, late 2024/2025). A deliberately "small" model from HuggingFace's TB team, overtrained on curated data and positioned as a state-of-the-art small LM; its paper reports strong ARC-class results. Included as the community's favorite open small model and as a test of whether paper benchmarks transfer to a strict user-side harness (they did not — §7). Technical report: [arXiv:2502.02737](https://arxiv.org/abs/2502.02737). Weights: [HuggingFaceTB/SmolLM2-1.7B-Instruct-GGUF](https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct-GGUF).

Two further models were measured and set aside, with reasons: **Qwen3-1.7B** (thinking-trained; its chain-of-thought habit makes raw-completion benchmarks unrepresentative — Q4_K_M 63.5%, Q8_0 67.0%) and **Ministral-3B** (over the speed ceiling and less accurate than the 1.5Bs; the cautionary data point of §7).

2.3 Quantizations tested

Twelve configurations across three families, all first-party GGUF sources:

Family	Configurations tested
Qwen2.5-1.5B-Instruct	Q4_0, Q5_0, Q5_K_M, Q6_K (official)
glm-edge-1.5b-chat	Q4_1, Q5_0, Q5_1, Q5_K_M, Q6_K (official)
SmolLM2-1.7B-Instruct	Q4_K_M (official), Q5_0, Q5_K_M, Q6_K (self-quantized from official safetensors)

Excluded during the study: Ministral-3B and Gemma-2-2b (over the speed ceiling for negligible or negative accuracy gain; Ministral scored ~54% while running at ~12 t/s — see §7), Qwen3-1.7B (thinking-trained; measured but set aside for the 2024-vs-2025 comparison, Q4_K_M 63.5% / Q8_0 67.0%), and SmolLM2's Q5_1 (redundant with the encoding grid).

2.4 Protocols

Speed. llama-bench token-generation throughput (tg128, mean of 3 repeats; observed repeatability $\pm 0.7\%$). Prompt-processing throughput was excluded from analysis (-p 0). Early sessions inadvertently included the default pp512 test; this was corrected mid-study (see §8).

Live calibration. Bench throughput overstates interactive speed. A calibration factor of live/bench $\approx 0.72\text{--}0.86$ (point estimate 0.80) was established from three direct comparisons between bench tg128 and observed interactive token rates.

Accuracy. ARC-Challenge, 800 questions, fixed prompt template, single-letter answer enforced (no chain-of-thought), parsed strictly. Binomial 95% confidence interval at $n=800$ is ± 3.3 percentage points. All configurations used the identical question cache, enabling paired comparison. Prompt-token counts differ slightly across tokenizers (mean 66.3–69.7), which is expected and does not affect question equivalence.

Provenance. Only first-party GGUFs (model organization's own repos) or self-converted quantizations from official safetensors were used. SmolLM2's official repo ships only Q4_K_M; the Q5_0/Q5_K_M/Q6_K variants were converted locally from HuggingFaceTB/SmolLM2-1.7B-Instruct with a pinned llama.cpp converter, then quantized with llama-quantize . The full pipeline is reproducible from the companion repository (Codeberg): [dzaragoza/LLM-benchmarks](https://codeberg.org/dzaragoza/LLM-benchmarks).

3. Predicting speed: the bandwidth formula

3.1 The formula

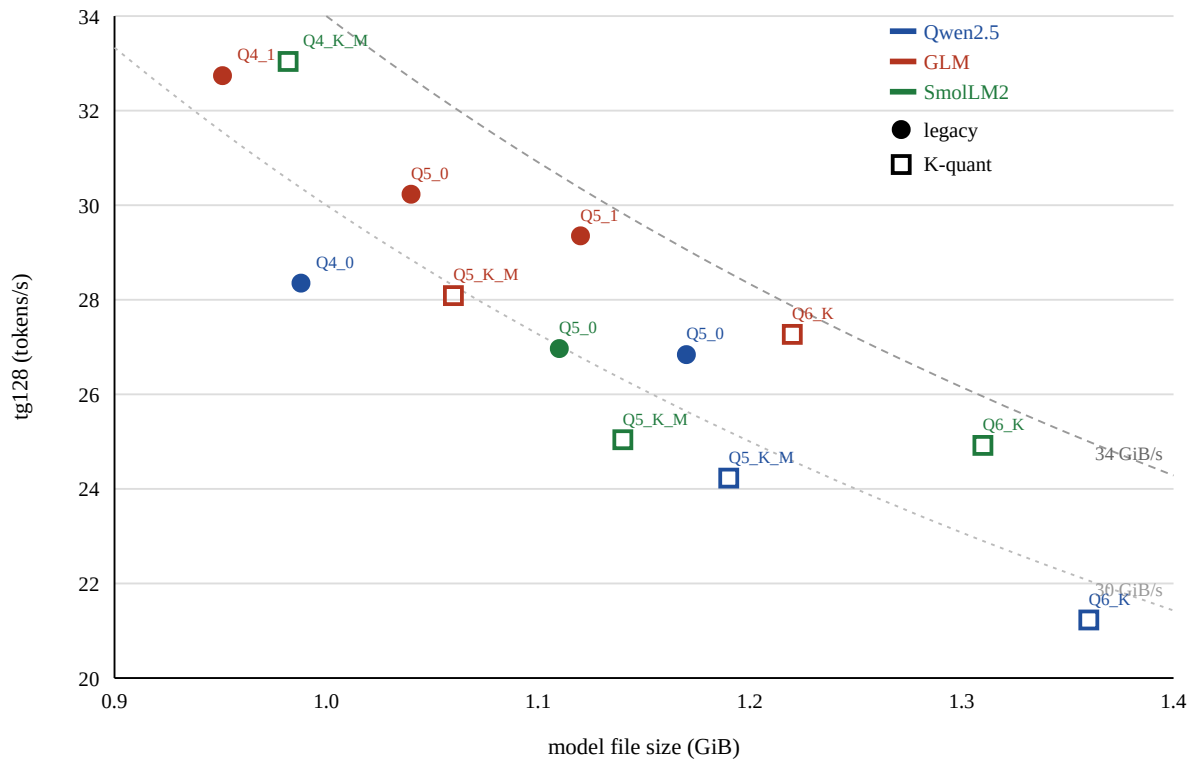
Token generation on an LLM is memory-bandwidth-bound: every generated token requires streaming the model weights once. The natural model is:

$$\boxed{\text{predicted tokens/second} \approx \text{effective bandwidth} \div \text{model file size}}$$

Fitting our twelve measurements gives an **effective bandwidth of ~34 GiB/s** for this device in legacy encodings, with two systematic deviations:

- **Family factor:** Qwen2.5 configurations imply ~30–31 GiB/s even in legacy encodings — a ~10% family-level penalty, possibly related to its large (152k) vocabulary and per-token output-head compute.
- **Encoding factor:** K-quant encodings imply a further ~2 GiB/s reduction (§5).

Figure 1. All twelve configurations: file size vs. generation throughput. Dashed curves mark constant implied bandwidth (34 and 30 GiB/s). Circles are legacy encodings, open squares are K-quants.



Plotted values (size GiB, tg128 t/s): Qwen2.5 — Q4_0 (0.99, 28.35), Q5_0 (1.17, 26.84), Q5_K_M (1.19, 24.23), Q6_K (1.36, 21.23); GLM — Q4_1 (0.95, 32.74), Q5_0 (1.04, 30.23), Q5_1 (1.12, 29.35), Q5_K_M (1.06, 28.09), Q6_K (1.22, 27.27); SmoLLM2 — Q4_K_M (0.98, 33.04), Q5_0 (1.11, 26.97), Q5_K_M (1.14, 25.04), Q6_K (1.31, 24.92).

3.2 Accuracy of the formula

Predictions were made *before* each measurement (logged with dates in the companion notebook). Results:

- Throughput predictions landed within $\pm 10\%$ for legacy encodings, $\pm 15\%$ including family and encoding corrections.
- Bench-to-live calibration: live speed $\approx 0.80 \times$ bench tg128, range 0.72–0.86.

- Repeatability of the bench itself: $\pm 0.7\%$ (Q4_K_M measured twice, 32.80 and 33.04).

Practical form. For this machine: **live t/s $\approx 26 \div$ file size in GiB** (for typical families; use ~ 23 for Qwen2.5-family). Inverted, this converts any speed preference into a size budget (§6).

3.3 The ceiling

Inverting the author's comfort threshold of 20 tokens/second live gives a model budget of **~ 1.3 GiB** (error band 1.22–1.47; ~ 1.15 for a slow family). Measured brackets: 1.19 GiB ran at 26 t/s live (clear); 1.36 GiB ran at ~ 17 t/s (over). The conservative pre-commitment rule: bench tg128 ≥ 26 before adopting a configuration.

4. Quantization and accuracy: the recovery effect

ARC-Challenge results (n=800; binomial 95% CI ± 3.3 pp on every score; bench repeatability $\pm 0.7\%$; live estimates $\pm 10\%$):

Configuration	ARC (± 3.3 pp)	Live t/s (est., $\pm 10\%$)	In ceiling?
Qwen2.5 Q5_0	75.2%	~ 21	✓
Qwen2.5 Q6_K	75.0%	~ 17	✗
Qwen2.5 Q5_K_M	74.6%	~ 19	⚠
Qwen2.5 Q4_0	71.8%	~ 32	✓
GLM Q5_0	67.4%	~ 24	✓
GLM Q6_K	66.4%	~ 21	✓
GLM Q5_K_M	65.4%	~ 22	✓
GLM Q4_1	63.9%	—	✓
Qwen3-1.7B Q8_0	67.0%	—	✓
Qwen3-1.7B Q4_K_M	63.5%	—	✓
SmolLM2 Q6_K	55.2%	~ 20 (line)	⚠
SmolLM2 Q5_0	54.9%	~ 21	✓
SmolLM2 Q4_K_M	52.0%	~ 26	✓
SmolLM2 Q5_K_M	49.6%	~ 20	✓

Finding 1: quant recovery is consistent. Moving from Q4 to Q6 improves ARC by +2.5 to +3.2 points in *all three families* (Qwen2.5: 71.8 $\rightarrow$ 75.0; GLM: 63.9 $\rightarrow$ 66.4; SmolLM2: 52.0 $\rightarrow$ 55.2). Authors ship Q4 by default for download size; readers should know it quietly costs ~ 3 points.

Finding 2: the recovery curve has a flat top. For Qwen2.5, Q4 → Q5 captures +3.4 points; Q5 → Q6 adds nothing measurable (75.2 → 75.0). If Q5 is available in a fast encoding, it is the accuracy-per-byte optimum — which is exactly how our champion configuration was found.

5. Encoding: the suffix matters (for speed)

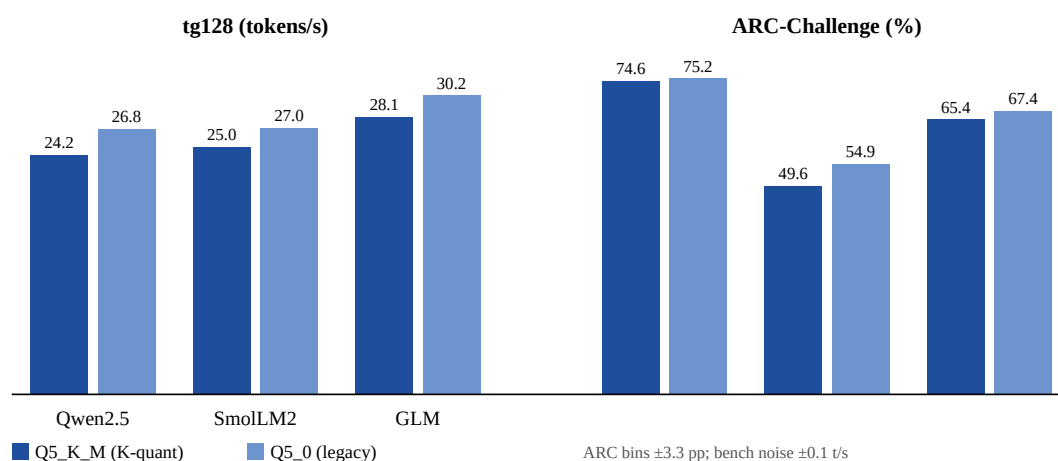
The GGUF quantization zoo divides into legacy formats (Q4_0, Q5_0, ...), K-quants (Q5_K_M, Q6_K, ...), and i-quants (untested here). A natural suspicion is that the suffix is mere bookkeeping. It is not — at least not on this backend.

Speed. At essentially equal file size, Q5_0 decodes faster than Q5_K_M in every family tested (bench noise ± 0.1 t/s):

Family	Q5_0 (t/s)	Q5_K_M (t/s)	Penalty
Qwen2.5	26.84 $\pm$ 0.04	24.23 $\pm$ 0.06	−11%
SmolLM2	26.97 $\pm$ 0.02	25.04 $\pm$ 0.02	−7%
GLM	30.23 $\pm$ 0.02	28.09 $\pm$ 0.00	−7%

Bench noise is ± 0.1 t/s; these gaps are measurements, not noise. Implied bandwidth clusters cleanly: legacy encodings 29.9–32.9 GiB/s, K-quants 28.5–29.8 — with one anomaly: GLM's Q6_K showed *no* penalty (33.3 GiB/s), unexplained and flagged as such.

Figure 2. The encoding trade-off in one view: Q5_0 (legacy, light) vs Q5_K_M (K-quant, dark) per family — speed on the left panel, accuracy on the right. The speed gap appears in all three families; the accuracy gap does not.



Plotted values — tg128 (Q5_K_M, Q5_0): Qwen2.5 (24.23, 26.84), SmolLM2 (25.04, 26.97), GLM (28.09, 30.23).
ARC % (Q5_K_M, Q5_0): Qwen2.5 (74.6, 75.2), SmolLM2 (49.6, 54.9), GLM (65.4, 67.4).

Accuracy. The compensating benefit is not evident. Q5_0 vs Q5_K_M on ARC: +0.6 points for Qwen2.5 (noise), +2.0 for GLM (edge of noise), +5.3 for SmolLM2 in favor of Q5_0 — the only gap outside the ± 3.3 -point band, and in the *wrong direction* for the K-quant. Single-family significance; suggestive, not conclusive.

Conclusion. In this study, K-quant at Q5 were slower and never meaningfully smarter. Legacy Q5_0 is the speed-quality optimum where available. The mechanism of the speed penalty (Vulkan shader paths for superblock decoding) is not investigated here; we note that the penalty is family-dependent in magnitude, and that one earlier stage of this study wrongly concluded the effect was absent (see §8).

6. The speed threshold is a preference — here is the dial

The author's operating rule was " ≥ 20 tokens/second live feels good." That is a preference, not a finding, and this report declines to dress it as one. The reasoning behind it: silent reading averages 200–250 wpm (≈ 5 – 6 t/s) and fast readers reach ≈ 9 – 10 t/s, so even 10 t/s outpaces reading. The 20 t/s pick is about *conversation rhythm* — a 200-token answer takes 10 s at 20 t/s but 20 s at 10, and the wait compounds over a session.

Because the formula converts speeds to sizes in one line, the reader can set their own threshold:

If you're happy with...	Model budget (typical family)	What it buys
10 t/s live	~ 2.6 GiB	3B-class at Q4 — "read it when it's done"
15 t/s	~ 1.7 GiB	2B at Q4–Q5, or 1.7B at Q8
20 t/s (author's pick)	~ 1.3 GiB	1.5–1.7B at Q5 — chat-rhythm sweet spot
25 t/s	~ 1.0 GiB	1.5B at Q4–Q5
30 t/s	~ 0.87 GiB	"must feel instant"

Error bars: $\pm 10\%$ on the formula; subtract $\sim 10\%$ of budget for Qwen2.5-family models. The author's recommendation is 20 t/s; the reader's mileage is explicitly allowed to vary.

7. Surprises and cautionary tales

SmolLM2's paper-to-harness gap. SmolLM2-1.7B-Instruct is marketed with strong ARC results, and entered this study as a predicted contender against Qwen2.5's 71.8%. It scored 52.0% — roughly 20 points below expectation — despite being the fastest family measured (33.4 GiB/s implied). Its published numbers come from a different evaluation protocol (ours enforces letter-only

answers without chain-of-thought; its answers also ran visibly longer, per timing). We do not claim SmolLM2 is a bad model; we claim **paper benchmarks do not transfer across harnesses**, and user-side evaluation of the exact serving stack you will run is the only reliable guide. Incidentally, this result strengthened the study's earlier observation that 2024-era non-thinking models at this scale outperform newer ones on raw-completion evals.

The over-ceiling tier is not worth it here. Ministral-3B ran at ~12 t/s (over the line) and scored ~54% — slower *and* less accurate than every in-ceiling 1.5B configuration. Speed ceilings exist partly because bigger is not automatically smarter at this scale and quantization level.

8. Corrections log

A study that never corrects itself isn't measuring. Predictions and conclusions that failed, preserved on purpose:

1. **"Encoding exonerated" (Session 23) — wrong.** An early analysis concluded the quant suffix had no measurable effect, based on GLM's Q6_K showing no penalty. The Qwen2.5 Q5_0-vs-Q5_K_M pair (Session 24) falsified this, and the cross-family grid (Session 26) established the corrected statement: legacy encodings decode ~7–11% faster, family-dependent, with GLM's Q6_K as an unexplained exception.
2. **SmolLM2 ARC "coin flip vs 71.8%" — badly wrong.** Actual: 52.0%. The failure mode (trusting paper-highlight evals across harness change) is itself a finding, reported in §7.
3. **SmolLM2 Q5/Q6 speed predictions (~28–29 t/s) — ~11% optimistic,** by assuming the K-quant penalty would not apply to a "fast family." It applied.
4. **Protocol correction:** omitting `-p` from `llama-bench` does not skip prompt processing (the default `pp512` runs anyway); `-p 0` is required. Mid-study fix, uniform thereafter.
5. **Quant recovery predictions — right, three times** (+2.5 to +3.2 points, predicted 72–73.5 / 65–67 / on-record band). Recorded for balance.

9. Recommendations

This section is written to stand alone: a reader who skips the rest of the report should still leave with usable guidance. Each point links to the section containing the supporting data.

1. **Default pick: Qwen2.5-1.5B-Instruct Q5_0** — 1.5B parameters, Q5 quantization level, legacy Q5_0 encoding; 1.17 GiB file, ~21 tokens/s live, 75.2% on our ARC protocol. This configuration is the accuracy-per-byte optimum found in this study ([§4, Table](#)). **Corollary — re-quantizing is worth it when the author's repo doesn't ship your size:** the author's official repo provides only Q4_K_M and Q8_0, so our champion file was created by converting from the official safetensors to Q5_0. If the size you need is missing from the first-party repo, re-quantize from the original weights (pinned converter, [Appendix A](#)) or use an alternative repo that ships it — the quantization ladder is a dial you are allowed to turn.

2. **Size budget: ~1.3 GiB for a 20 tokens/s live experience** (§6 has the table for other thresholds; verification rule: `llama-bench tg128 ≥ 26` before adopting).
3. **The limit is memory bandwidth, not computing power.** This machine's iGPU achieves ~34 GiB/s effective bandwidth against a theoretical maximum of 51.2 GiB/s — DDR4-3200, dual channel (128-bit bus): $3200 \text{ MT/s} \times 16 \text{ bytes} \approx 51.2 \text{ GB/s}$. Token generation streams the model weights from RAM once per token, so *any* hardware with the same memory gets roughly the same generation speed class, regardless of compute. To go faster, you need faster memory (or a smaller file) — a bigger GPU alone won't help generation unless it also has more bandwidth (§3 derives and calibrates this).
4. **Prefer the GPU (Vulkan) backend, and here is why:** in our measurements the iGPU processed prompts ~2× faster than the CPU while generation speed favored the CPU (41.2 vs 28.4 t/s on the reference model) or ran at parity. Time-to-first-token — dominated by prefill — is the responsiveness you feel on every message, so the GPU's compute advantage in prefill buys the better overall experience, and the generation-speed gap is closed by model selection rather than hardware (§1.3 has the measurements).
5. **Prefer legacy encodings where a size you want exists** (Q5_0 over Q5_K_M): the K-quant suffix cost 7–11% generation speed in all three families tested, with no compensating accuracy gain (§5, Figure 2 has the cross-family data).
6. **Never trust paper benchmark numbers across harnesses** — one popular model scored ~20 points below its published impression under our strict letter-answer protocol. Run your own 15-minute evaluation (our harness and question cache are in the [companion repository on Codeberg](#)).

10. Limitations and future work

Single benchmark (ARC-Challenge), single machine, one backend (Vulkan, one build), n=800 with ±3.3-point bins, self-made quantizations for one family, live-calibration from three sessions, encoding mechanism unprofiled, i-quants untested, and the GLM Q6_K anomaly unexplained. A natural extension is perplexity measurement (`llama-perplexity`, minutes per model on the existing GGUFs): whether PPL tracks our ARC-based quant-recovery and encoding findings would test if the two instruments agree on where quality is lost. Each is one weekend of work away from closure; none, we believe, is load-bearing for the recommendations above at this hardware class.

References

1. Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115 (2024).
<https://arxiv.org/abs/2412.15115> — model weights: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF>
2. Z.ai (Zhipu AI). *GLM-Edge: Fundamental Models for On-Device Applications*. (2025).
Technical report link via <https://huggingface.co/zai-org/glm-edge-1.5b-chat> — model weights:

<https://huggingface.co/zai-org/glm-edge-1.5b-chat-gguf>

3. Allal, L. A., et al. *SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model*. arXiv:2502.02737 (2025). <https://arxiv.org/abs/2502.02737> — model weights: <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct-GGUF>
4. llama.cpp project (build b10964, commit b29c606e2). <https://github.com/ggml-org/llama.cpp>
5. ARC-Challenge benchmark: Clark, P., et al. *From 'F' to 'A' on the New York State Regents Examinations*. (2016). <https://allenai.org/data/arc>

Appendix A: Reproducibility

All commands, versions, and the session-by-session lab notebook (with pre-registered predictions and their outcomes) are in the companion repository: codeberg.org/dzaragoza/LLM-benchmarks. Key pins: llama.cpp b10964 (b29c606e2); ARC-Challenge test split (cached, 800 questions); SmolLM2 conversions from official safetensors with pinned converter.

Appendix B: Session log summary

Twenty-seven sessions, 2026-09-14 through 2026-09-20: hardware discovery, formula calibration, candidate screening and exclusions, quant ladder, encoding grid, threshold reframing, and the final eight-model overnight evaluation. Every measurement was preceded by a written prediction; every prediction is graded in §8.

September 2026 · DOI: [10.5281/zenodo.22855666](https://doi.org/10.5281/zenodo.22855666) · License: CC BY 4.0 · Code: codeberg.org/dzaragoza/LLM-benchmarks